



MASTG

- 0294: `SecureOn` Not Used to Prevent Screenshots in Compose Dialogs
- MASTG-TEST-0315: Sensitive Data Exposed via Notifications
- MASTG-TEST-0316: App Exposing User Authentication Data in Text Input Fields
- MASTG-TEST-0320: WebViews Not Cleaning Up Sensitive Data
- MASTG-TEST-0334: Native Code Exposed Through WebViews
- MASTG-TEST-0340: References to Overlay Attack Protections

MASVS-CODE

MASVS-RESILIENCE

L1

L2

ANDROID

MASWE-0118

TEST



MASTG-TEST-0320: WebViews Not Cleaning Up Sensitive Data

Overview

This test verifies whether the app cleans up sensitive data used by WebViews. Apps can enable several specific storage areas in their WebViews and not clean them up properly, leading to sensitive data being stored on the device longer than necessary. For example:

- Not calling `WebView.clearCache(includeDiskFiles = true)` when:
 - `WebSettings.setAppCacheEnabled()` is enabled,
 - or `WebSettings.setCacheMode()` is any value other than `WebSettings.LOAD_NO_CACHE`.
- Not calling `WebStorage.deleteAllData()` when:
 - `WebSettings.setDomStorageEnabled` is enabled.
- Not calling `WebStorage.deleteAllData()` when:
 - `WebSettings.setDatabaseEnabled()` is enabled.
- Not calling `CookieManager.removeAllCookies(ValueCallback<Boolean> ...)` when:
 - `CookieManager.setAcceptCookie()` is not explicitly set to `false` (default is set to `true`).

This test uses dynamic analysis to monitor the relevant API calls and file system operations. Regardless of whether the app uses these APIs directly, WebViews may use them internally when rendering content (e.g., JavaScript code using `localStorage`). So tracing calls to APIs such as `open`, `openat`, `opendir`, `unlinkat`, etc., can help identify file operations in the WebView storage directory.

Steps

- Install the app on a device ([Installing Apps](#)).
- Use [Method Tracing](#) to target WebView APIs for storage enablement and cleanup.
- Open the app.
- Use the app extensively to ensure that all relevant WebViews are covered and that sensitive data is loaded into them. Ensure you keep a list of the sensitive data you expect to be cleaned up.
- Close the app.
- Use [Host-Device Data Transfer](#) to pull the contents of the `/data/data/<app_package>/app_webview/` directory or simply search for the sensitive data used in the WebView within that directory.

Observation

The output should include:

- The list of WebView storage enablement APIs used.
- The list of WebView storage cleanup APIs used.
- The list of sensitive data expected to be cleaned up.
- The result of searching the contents of the `/data/data/<app_package>/app_webview/` directory for the sensitive data used in the WebView after closing the app.

Evaluation

The test case fails if the app still has sensitive data on the `/data/data/<app_package>/app_webview/` directory after the app is closed. This could be due to the app not calling the relevant cleanup APIs after using the WebView.

The test passes if all sensitive data used by the WebView is properly cleaned up using the relevant APIs, and no sensitive data remains in the `/data/data/<app_package>/app_webview/` directory after closing the app.

Note

It can be challenging to determine whether the right cleanup APIs were called for the enabled storage areas. [WebViews](#) describes the storage areas used by WebViews and the challenges of evaluating their cleanup.

Additional Guidance:

If you need more introspection during runtime, you can rerun the test with additional tracing of file system operations in the WebView storage directory. See [Monitor File System Operations in WebViews](#).

Best Practices

MASTG-BEST-0028: WebViews Cache Cleanup

Demos

MASTG-DEMO-0082: WebView WebStorage Cleanup